Research Progress Review

Vortex-based Microarchitectural Benchmarking: Scheduler & Cache

4/17

1. 핵심 업데이트
2. 결정 사항
3. 다음 액션

Agenda

01 Configuration 선정

02 GTO/RR 구현

03 논문 리뷰 : CAWA

04 논문 리뷰 : iPAWS

[SECTION]

[Configuration 선정]

진행 현황

Configuration 선정 배경

Table 1: GPGPU-Sim Configuration

# Compute Units	30
Wavefront Size	32
SIMD Pipeline Width	8
Number of Threads / Core	1024
Number of Registers / Core	16384
Shared Memory / Core	16KB
Constant Cache Size / Core	8KB
Texture Cache Size / Core	32KB, 64B line, 16-way assoc.
Number of Memory Channels	8
L1 Data Cache	32KB, 128B line, 8-way assoc. LRU
L2 Unified Cache	128k/Memory Channel, 128B line, 8-way assoc. LRU
Compute Core Clock	1300 MHz
Interconnect Clock	650 MHz
Memory Clock	800 MHz
DRAM request queue capacity	32
Memory Controller	out of order (FR-FCFS)
Branch Divergence Method	PDOM [12]
GDDR3 Memory Timing	t_{CL}=10 t_{RP}=10 t_{RC}=35 t_{RAS}=25 t_{RCD}=12 t_{RRD}=8
Memory Channel BW	8 (Bytes/Cycle)

CCWS 논문에서 사용된 config (GTX285)

GPU	G80	GT200	Fermi
Transistors	681 million	1.4 billion	3.0 billion
CUDA Cores	128	240	512
Double Precision Floating Point Capability	None	30 FMA ops / clock	256 FMA ops /clock
Single Precision Floating Point Capability	128 MAD ops/clock	240 MAD ops / clock	512 FMA ops /clock
Warp schedulers (per SM)	1	1	2
Special Function Units (SFUs) / SM	2	2	4
Shared Memory (per SM)	16 KB	16 KB	Configurable 48 KB or 16 KB
L1 Cache (per SM)	None	None	Configurable 16 KB or 48 KB
L2 Cache (per SM)	None	None	768 KB
ECC Memory Support	No	No	Yes
Concurrent Kernels	No	No	Up to 16
Load/Store Address Width	32-bit	32-bit	64-bit

Tesla 아키텍처 스펙(GTX285)

[Cache-Conscious Wavefront Scheduling]

white paper: [NVIDIA's Next Generation Compute Architecture: Fermi]

진행 현황

Configuration 선정 배경

Architecture	*NVIDIA Fermi GTX480*
Num. of SMs	*15*
Max. # of Warps per SM	*48*
Max. # of Blocks per SM	*8*
# of Schedulers per SM	*2*
# of Registers per SM	*32768*
Shared Memory	*48KB*
L1 Data Cache	*16KB per SM (8-sets/16-ways)*
L1 Inst Cache	*2KB per SM (4-sets/4-ways)*
L2 Cache	*768KB unified cache (64-sets/16-ways/6-banks)*
Min. L2 Access Latency	120 cycles
Min. DRAM Access Latency	220 cycles
Warp Size (SIMD Width)	*32 threads*

Table 1: GPGPU-sim configurations.

CAWS 논문에서 사용된 config (GTX480)

GPU	G80	GT200	Fermi
Transistors	681 million	1.4 billion	3.0 billion
CUDA Cores	128	240	512
Double Precision Floating Point Capability	None	30 FMA ops / clock	256 FMA ops /clock
Single Precision Floating Point Capability	128 MAD ops/clock	240 MAD ops / clock	512 FMA ops /clock
Warp schedulers (per SM)	1	1	2
Special Function Units (SFUs) / SM	2	2	4
Shared Memory (per SM)	16 KB	16 KB	Configurable 48 KB or 16 KB
L1 Cache (per SM)	None	None	Configurable 16 KB or 48 KB
L2 Cache (per SM)	None	None	768 KB
ECC Memory Support	No	No	Yes
Concurrent Kernels	No	No	Up to 16
Load/Store Address Width	32-bit	32-bit	64-bit

white paper: [NVIDIA’s Next Generation Compute Architecture: Fermi] GF100 (GTX480)

진행 현황

Configuration 선정 배경

Table 2: GPGPU CONFIGURATIONS

SMs	32
Warp Size	32
Warps Scheduling	Round-Robin
L1 Data Cache	16 KB, 128 B line, 16 MSHRs, Write through, 8 ways ($Pways \leq 4$),
L2 Unified Cache	4 MB, 128 B line, 16 MSHRs, Write back, 16 ways

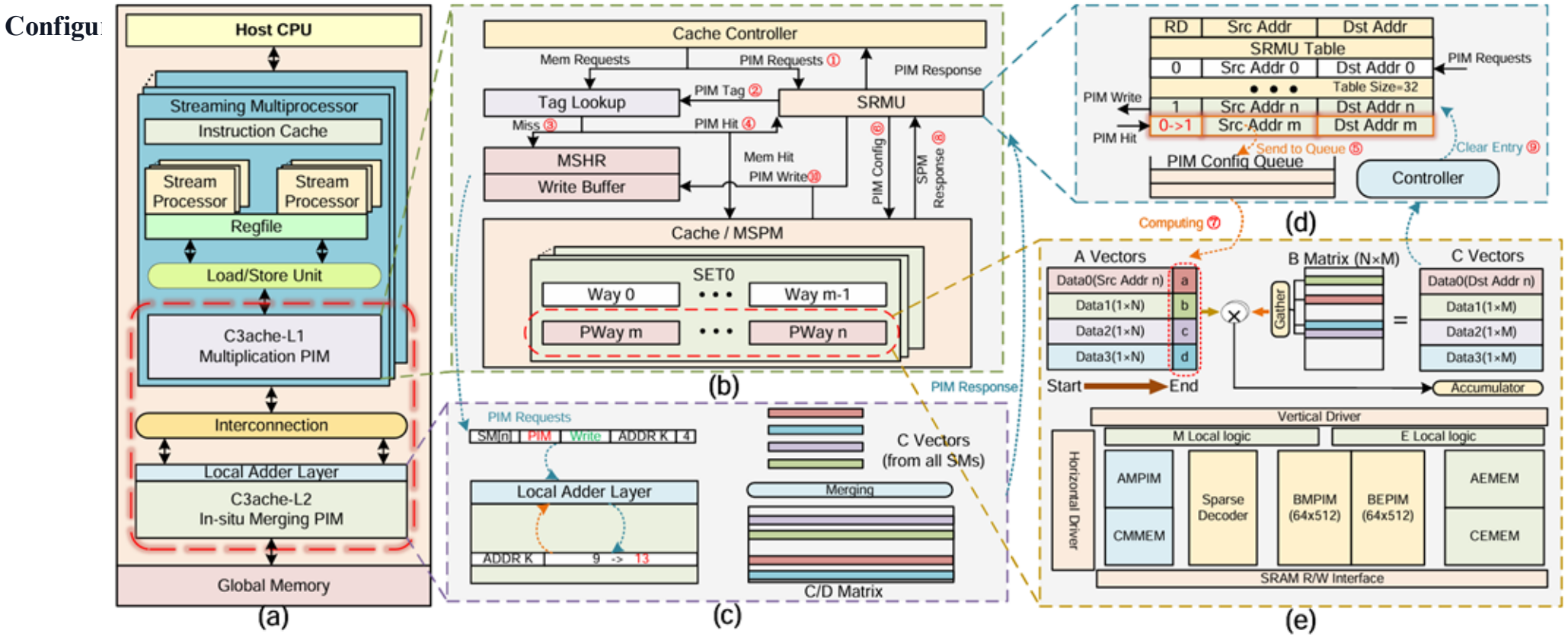
The **NVIDIA A100 Tensor Core GPU implementation** of the GA100 GPU includes the following units:

- 7 GPCs, 7 or 8 TPCs/GPC, 2 SMs/TPC, up to 16 SMs/GPC, 108 SMs
- 64 FP32 CUDA Cores/SM, 6912 FP32 CUDA Cores per GPU
- 4 Third-generation Tensor Cores/SM, 432 Third-generation Tensor Cores per GPU
- 5 HBM2 stacks, 10 512-bit Memory Controllers

Downscaled GPU on vortex. 1/12 TFLOPS, maintaining compute-memory ratio

C3ache, Xiaojie Li et al.

진행 현황



C3ache, Xiaojie Li et al.

진행 현황

Configuration 선정 배경



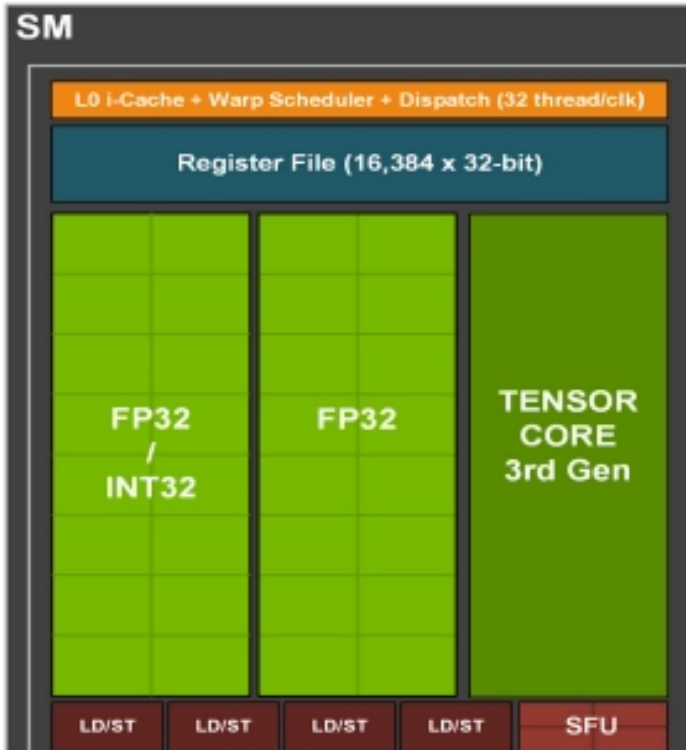
Note: The GA102 GPU also features 168 FP64 units (two per SM), which are not depicted in this diagram. The FP64 TFLOP rate is 1/64th the TFLOP rate of FP32 operations. The small number of FP64 hardware units are included to ensure any programs with FP64 code operate correctly, including FP64 Tensor Core code.

Figure 2. GA102 Full GPU with 84 SMs

- RTX 3090

진행 현황

Configuration 선정 배경



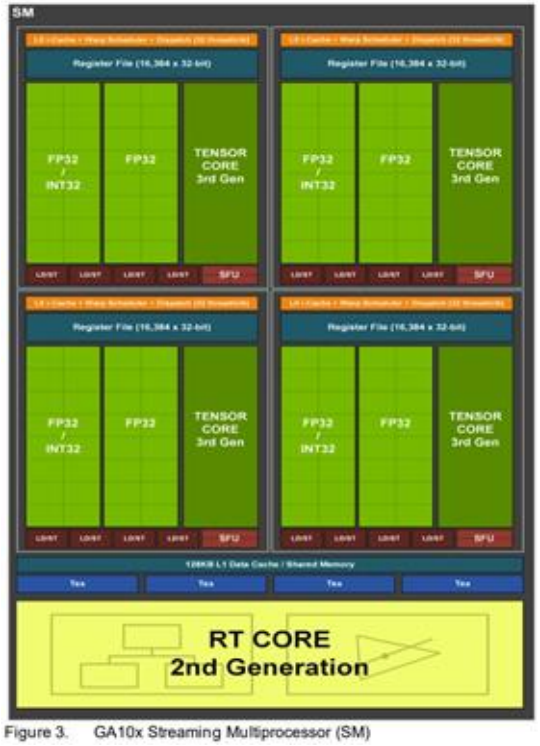


Figure 3. GA10x Streaming Multiprocessor (SM)

- 1 SM는 4개의 process block으로 구성.
- 각 process block은 32의 SIMD width를 가지고 있음.
- SM 내에서 128kB L1 data cache 공유
- 총 4(process block)* 32(FPU in process block)*82(SM) = 10496개의 FPU
- 128kB * 82 = 10496kB의 SharedMem/L1 data cache

  1 FPU당 1kB의 L1 data cache를 점유하고 있음

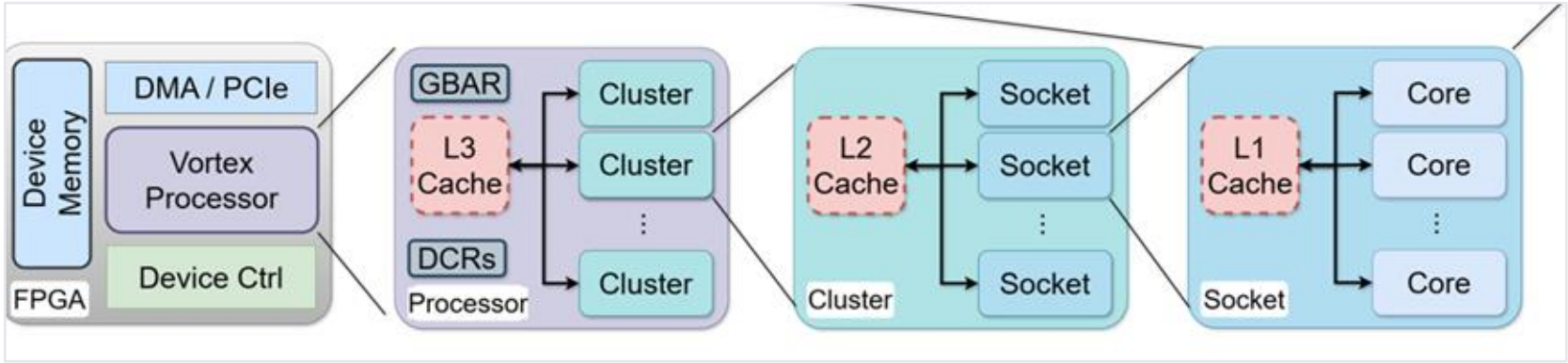
SM 및 하위 Process block 구성

진행 현황

Configuration 선정 배경

```
inline constexpr uint32_t NUM_SOCKETS      = UP(NUM_CORES / SOCKET_SIZE);

#ifndef SOCKET_SIZE
#define SOCKET_SIZE MIN(4, NUM_CORES)
#endif
```



한계점

- NVIDIA는 pure GPU
- Vortex는 CPU-modified GPU.

코어 구성부터 메모리 계층까지 설계 철학이 다르다.

Vortex: Extending the RISC-V ISA for GPGPU and 3D-GraphicsResearch

진행 현황

Configuration 선정 배경

cluster	1
core	16
lane(thread number)	16
warp size	32
L1 data cache (per socket(4 cores))	128KB, 128B line, 16 MSHR, Write through, 8 ways
L3 Unified cache (per cluster)	256kB, 128B line, 16 MSHR, Write back, 16 ways

1/20 downscaled from RTX 3090

진행 현황

Configuration 한계

--app=sgemm3 --cores=8 --threads=32

```
PERF: core7: lmem reads=0
PERF: core7: lmem writes=0
PERF: core7: lmem bank stalls=0 (utilization=0%)
PERF: core7: icache reads=4268
PERF: core7: icache read misses=357 (hit ratio=91%)
PERF: core7: icache mshr stalls=0 (utilization=100%)
PERF: core7: dcache reads=3180
PERF: core7: dcache writes=10328
PERF: core7: dcache read misses=3032 (hit ratio=4%)
PERF: core7: dcache write misses=10290 (hit ratio=0%)
PERF: core7: dcache bank stalls=4608 (utilization=74%)
PERF: core7: dcache mshr stalls=95307 (utilization=12%)
PERF: core7: coalescer misses=824 (hit ratio=93%)
PERF: core7: instrs=23693, cycles=189750, IPC=0.124864
```

한계점

- compute-bound로 여겨지는 sgemm3에서조차 잦은 L1 data cache miss로 인해 MSHR이 마비됨
- 시뮬레이션에 걸리는 시간이 너무 오래 걸림 (멀티코어를 사용할 수 있는지는 확인해봐야 함)

진행 현황

Configuration 한계

--app=sgemm3 --cores=8 --threads=32

```
PERF: l3cache reads=10711
PERF: l3cache writes=26909
PERF: l3cache read misses=10326 (hit ratio=3%)
PERF: l3cache write misses=26843 (hit ratio=0%)
PERF: l3cache bank stalls=8803 (utilization=81%)
PERF: l3cache mshr stalls=88189 (utilization=29%)
PERF: memory requests=37354 (reads=10295, writes=27059)
PERF: memory latency=243 cycles
PERF: memory bank stalls=13 (utilization=99%)
PERF: instrs=302022, cycles=261086, IPC=1.156791
make: Leaving directory '/home/ltj/2026urp/vortex/tests/opencl/sgemm3'
```

한계점

- compute-bound로 여겨지는 sgemm3에서조차 잦은 L1 data cache miss로 인해 MSHR이 마비됨
- 그에 반해 memory bank는 최대로 활용되고 있음을 볼 수 있음.

진행 현황

Configuration 한계

--cores=16 --warps=4 --threads=4

```
PERF: core0: lmem reads=512
PERF: core0: lmem writes=128
PERF: core0: lmem bank stalls=361 (utilization=63%)
PERF: core0: icache reads=11587
PERF: core0: icache read misses=529 (hit ratio=95%)
PERF: core0: icache mshr stalls=0 (utilization=100%)
PERF: core0: dcache reads=3357
PERF: core0: dcache writes=3715
PERF: core0: dcache read misses=1420 (hit ratio=57%)
PERF: core0: dcache write misses=3554 (hit ratio=4%)
PERF: core0: dcache bank stalls=0 (utilization=100%)
PERF: core0: dcache mshr stalls=0 (utilization=100%)
PERF: core0: coalescer misses=1504 (hit ratio=78%)
PERF: core0: instrs=10348, cycles=148752, IPC=0.069565
```

결론

- 상용 GPU를 모델링하여 현실성을 얻고자 하는 부분의 취지는 좋았으나, 저조한 L1 data cache hit ratio로 인해 memory bound, compute bound workload 모두에서 memory 병목이 일어나고 있음.
- 상용 GPU의 아키텍처 뿐만 아니라 메모리 시스템 규모 또한 반영해야 함

[SECTION]

[논문 리뷰]

Cache-Conscious Wavefront Scheduling. Timothy G. Rogers et al.

진행 현황

논문이 해결하고자 하는 병목

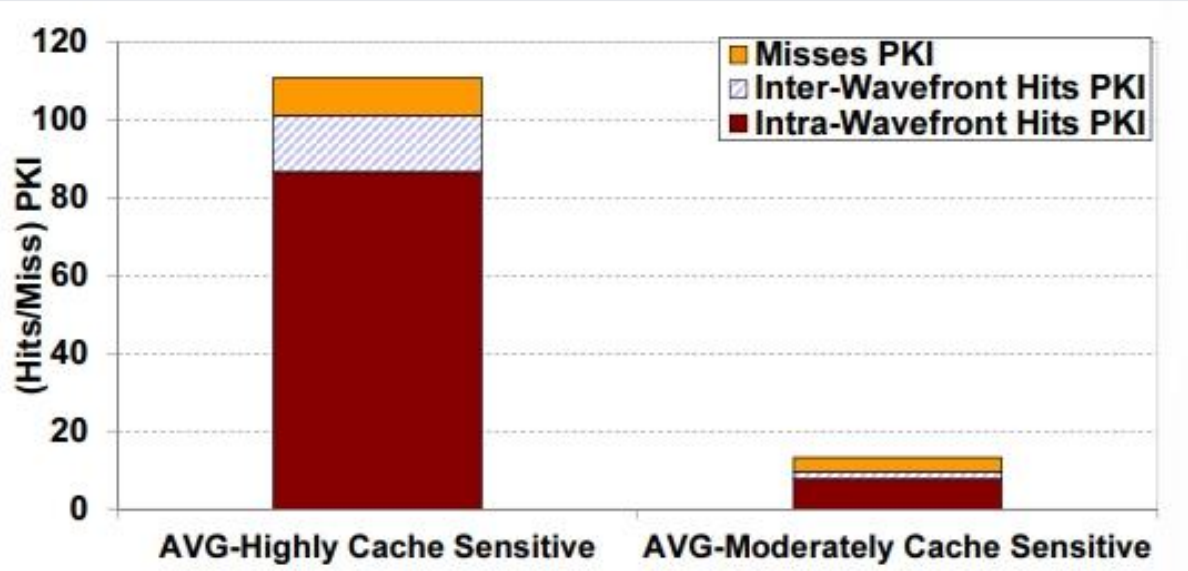
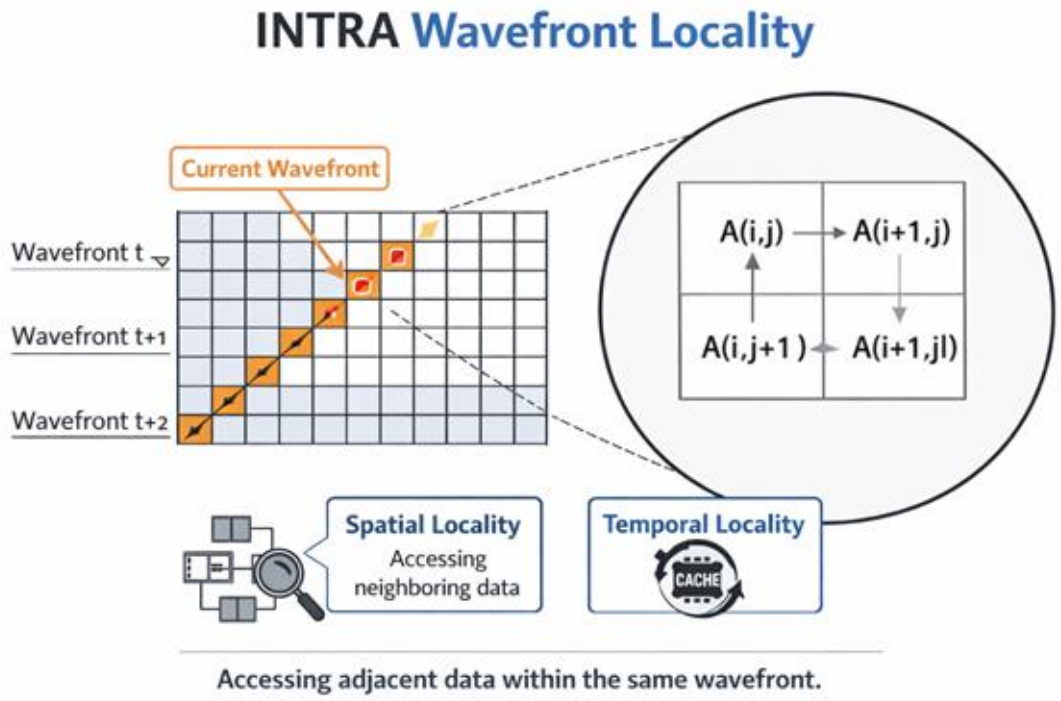
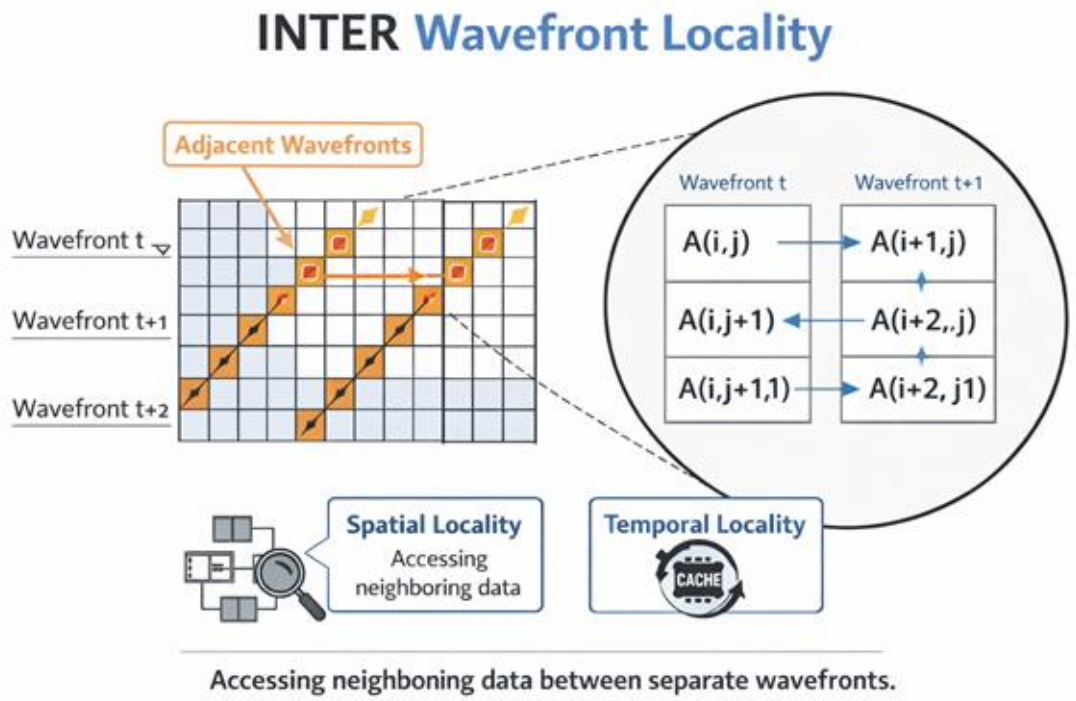


Figure 1: Average hits and misses *per thousand instructions* (PKI) using an unbounded L1 data cache (with 128B lines) on cache-sensitive benchmarks.

진행 현황

Intra/Inter wavefront locality





진행 현황

논문이 해결하고자 하는 병목

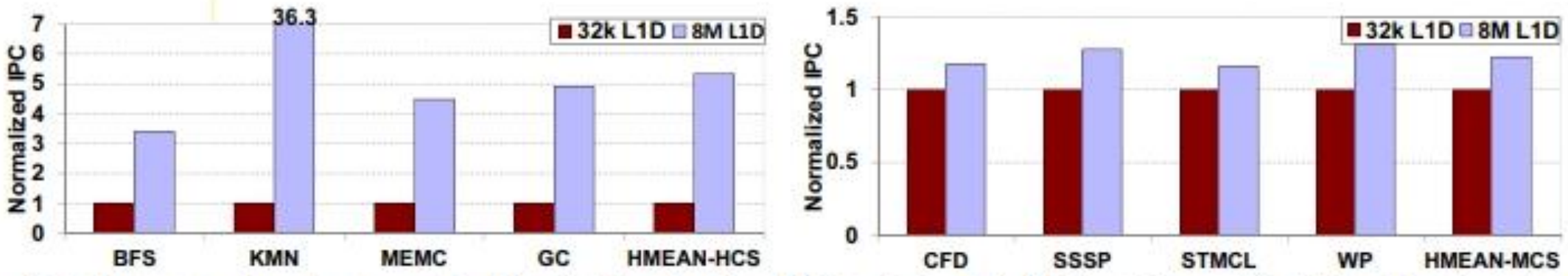


Figure 2: Performance using a loose round-robin scheduler at various L1D cache sizes for highly cache-sensitive (left) and moderately cache-sensitive benchmarks (right), normalized to a cache size of 32k. All caches are 8-way set-associative with 128B cache lines.

HCS 그룹

MCS 그룹

진행 현황

논문이 주목한 부분

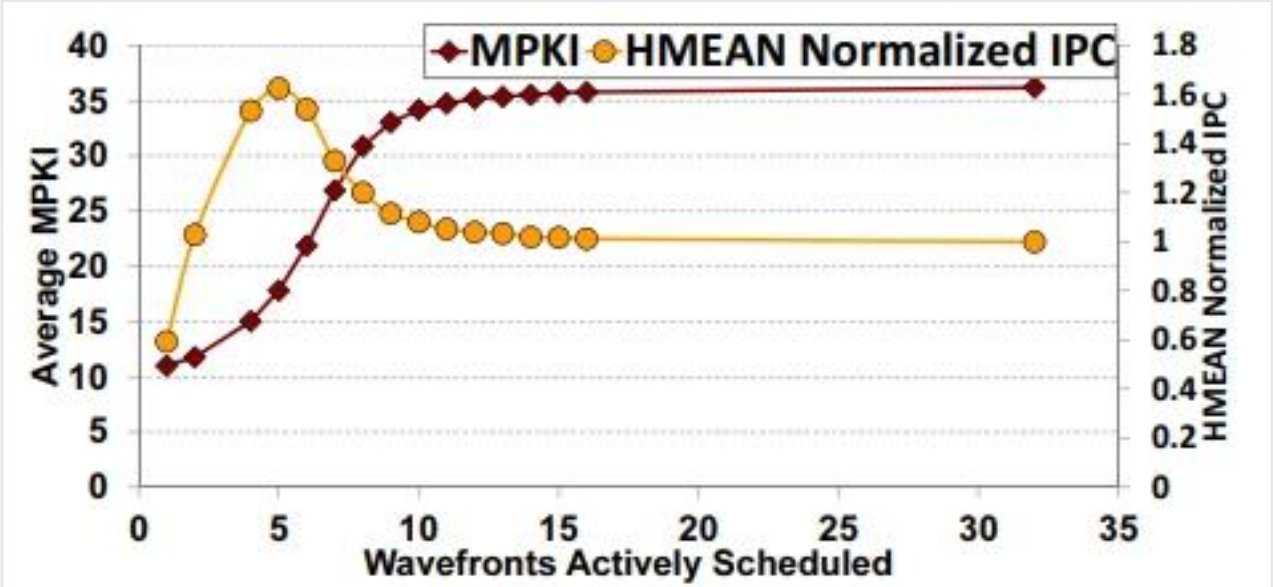


Figure 3: Average *misses per thousand instructions* (MPKI) and *harmonic mean* (HMEAN) performance improvement of HCS benchmarks with different levels of multithreading. *Instructions per cycle* (IPC) is normalized to 32 wavefronts.

진행 현황

논문이 주목한 부분

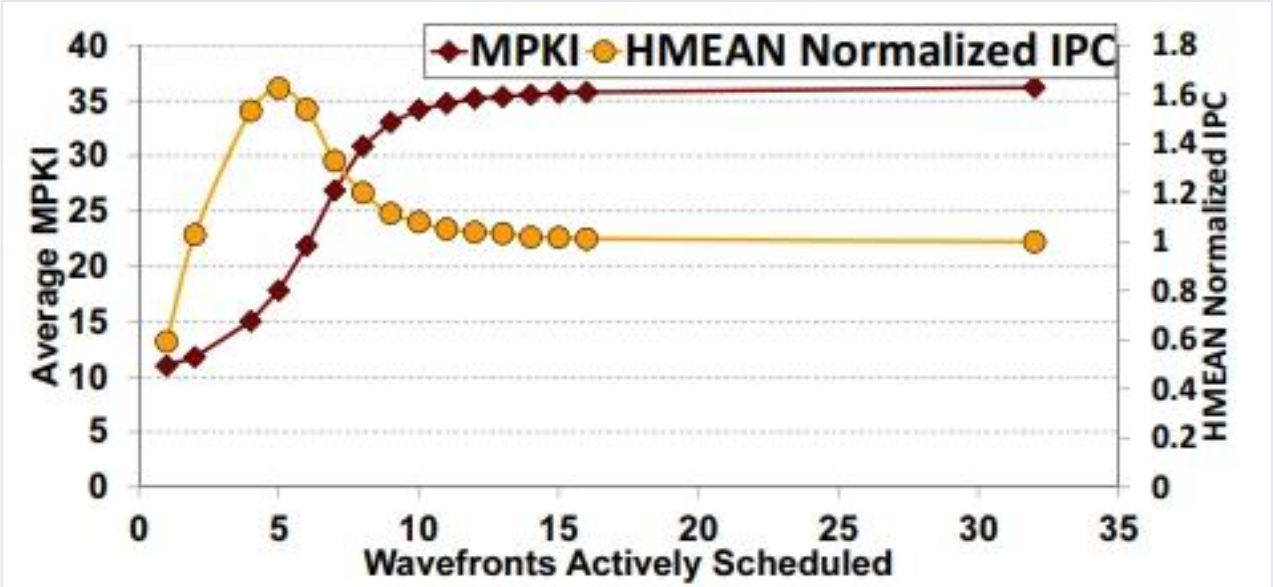


Figure 3: Average *misses per thousand instructions* (MPKI) and *harmonic mean* (HMEAN) performance improvement of HCS benchmarks with different levels of multithreading. *Instructions per cycle* (IPC) is normalized to 32 wavefronts.

진행 현황

실험환경

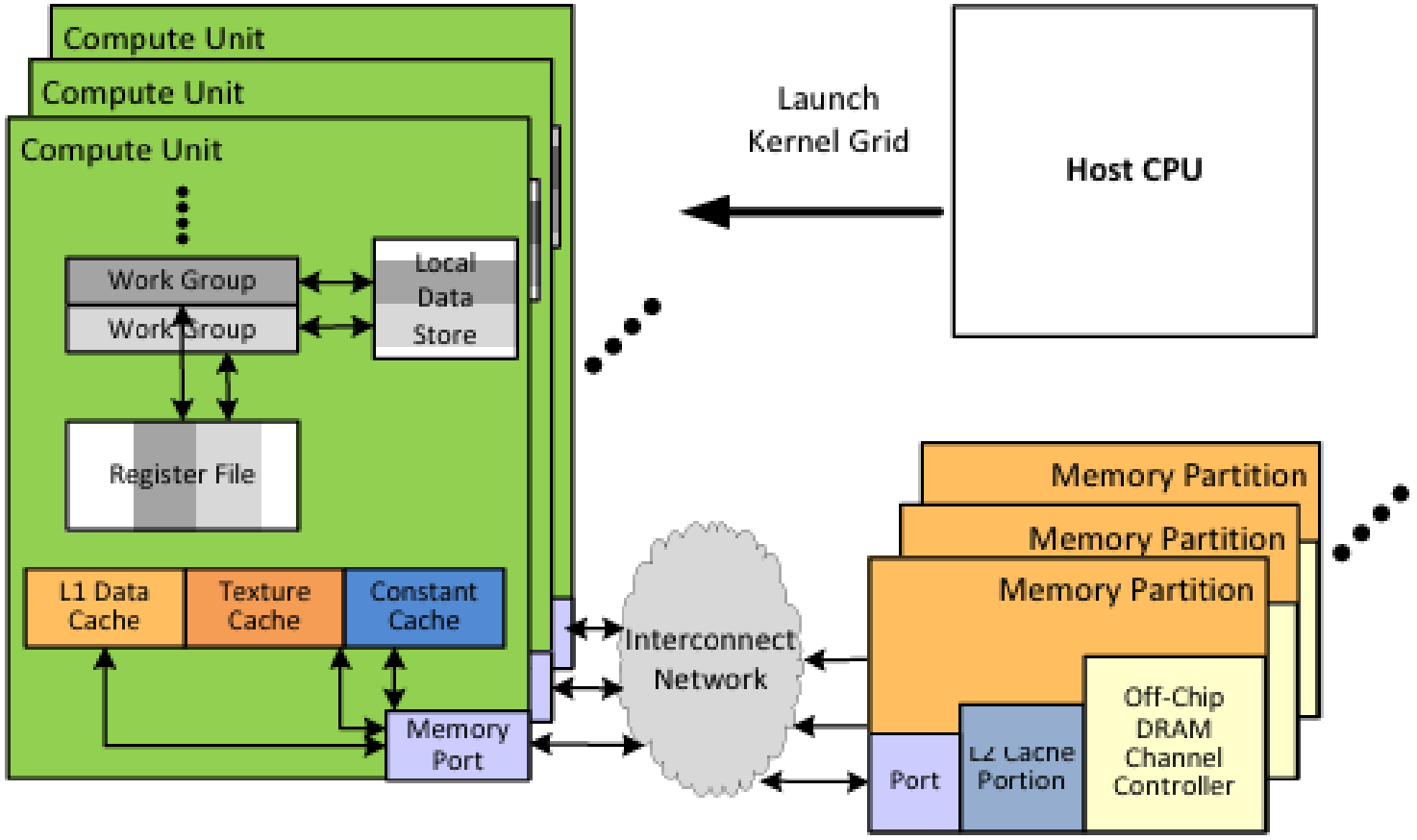
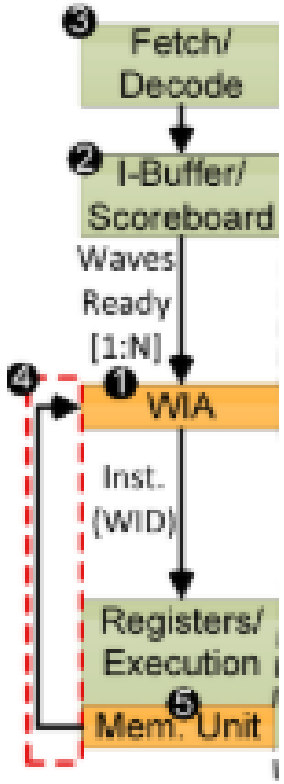
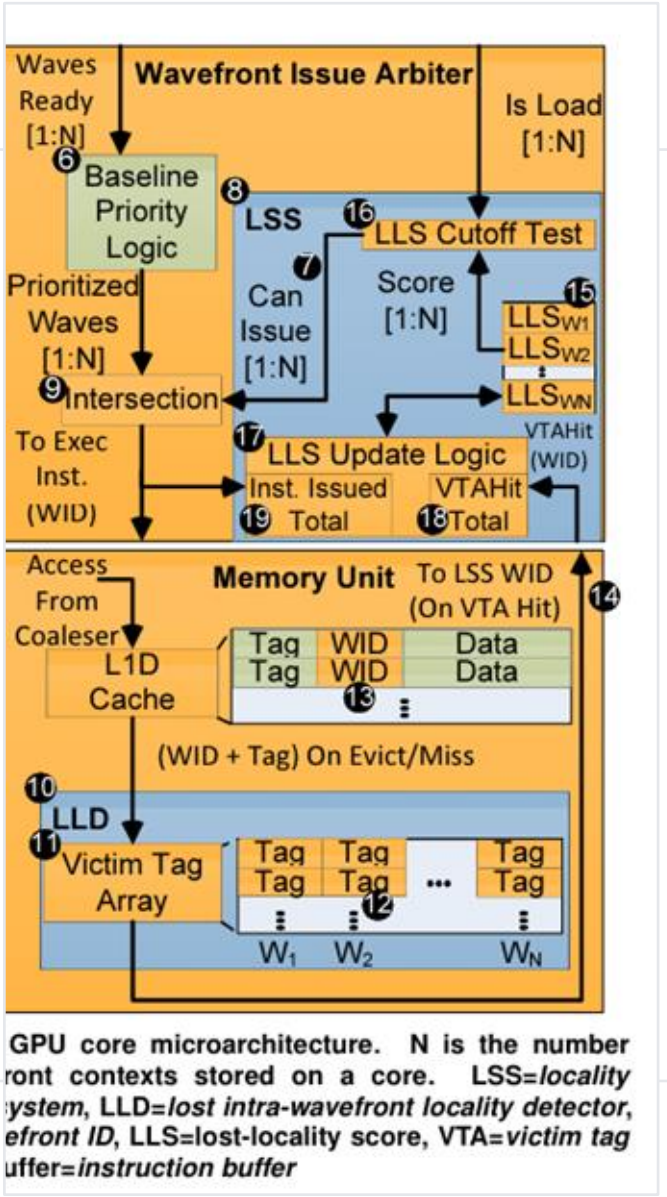


Figure 4: Overview of our GPU-like baseline accelerator architecture.

진행 현황

주요 변경 사항





GPU core microarchitecture. N is the number
ront contexts stored on a core. LSS=*locality*
ystem, LLD=*lost intra-wavefront locality detector*,
efront ID, LLS=lost-locality score, VTA=*victim tag*
uffer=*instruction buffer*

진행 현황

스케줄링 예시: **bfs**

Example 1 Example graph algorithm kernel run by each scalar thread.

```
int node_degree = nodes[thread_id].degree;
int thread_first_edge = nodes[thread_id].starting_edge;
for ( int i = 0; i < node_degree; i++ ) {
    edge_attribtes = edges[thread_first_edge + i];
    int neighbour_node_id = edge_attributes.node;
    int edge_weight = edge_attributes.weight;
    ...
}
```

진행 현황

Static Wavefront Limiting (SWL)

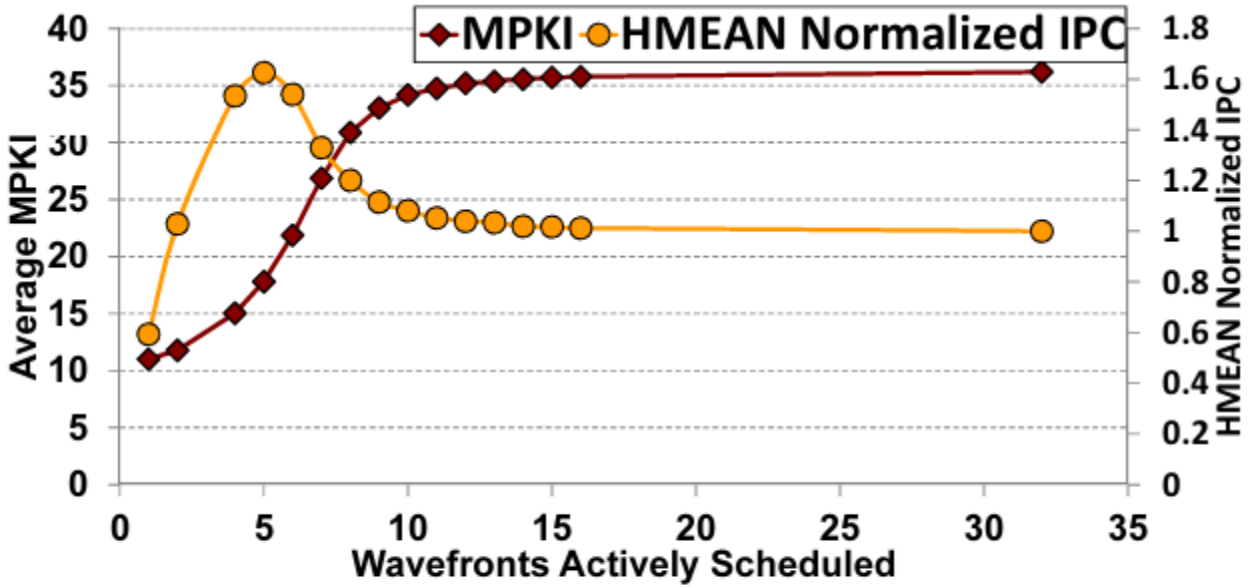
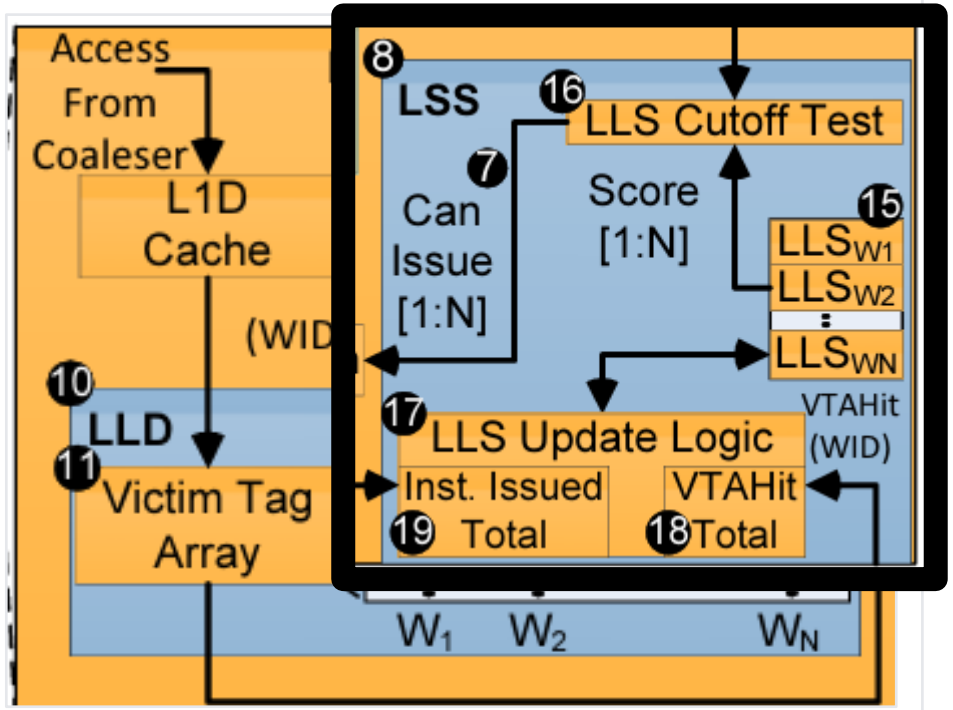


Figure 3: Average *misses per thousand instructions* (MPKI) and *harmonic mean* (HMEAN) performance improvement of HCS benchmarks with different levels of multithreading. *Instructions per cycle* (IPC) is normalized to 32 wavefronts.

진행 현황

CCWS - 목표와 high-level 구현



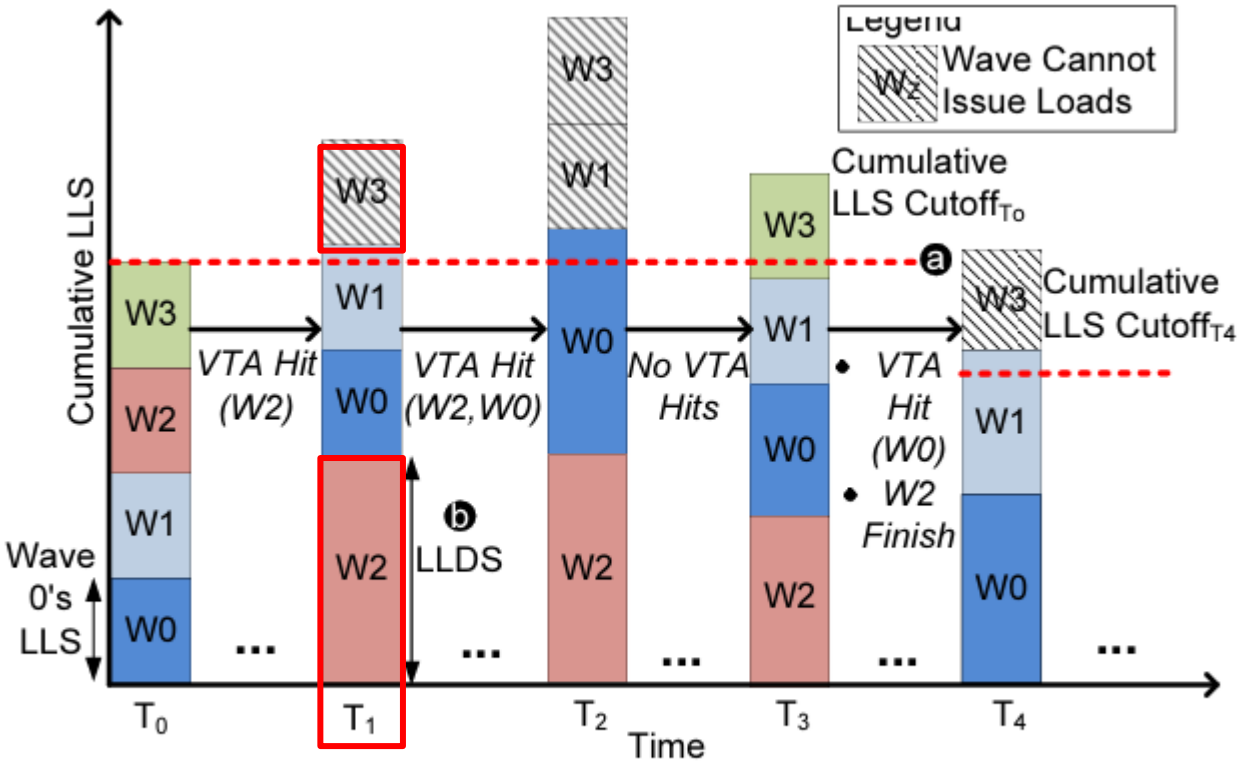
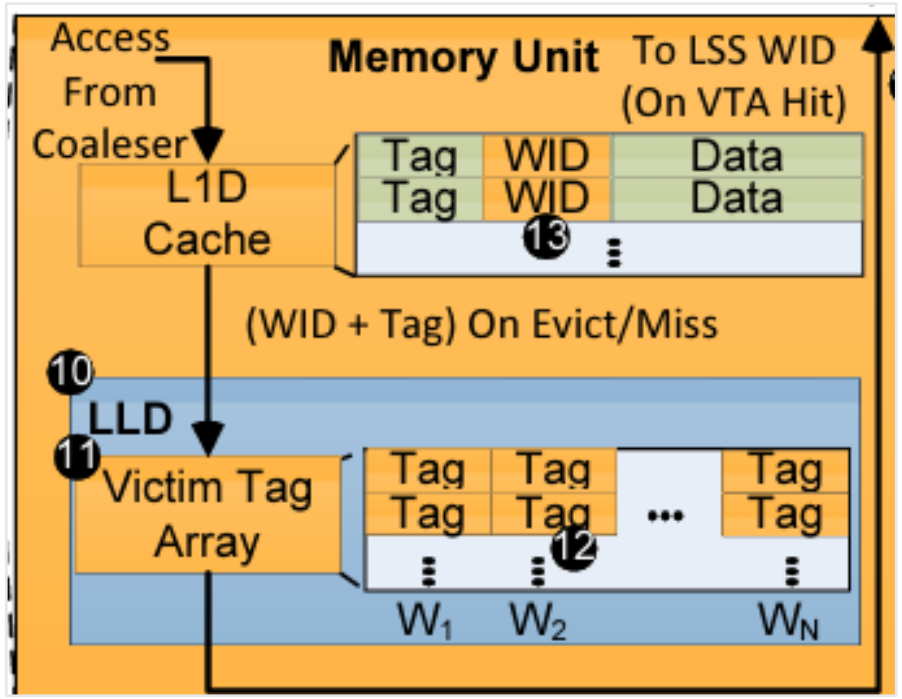


Figure 6: Locality scoring system operation example. LLS=*lost-locality score*, LLDS=*lost-locality detected score*

진행 현황

CCWS - Lost Intra-Wavefront Locality Detector(LLD)



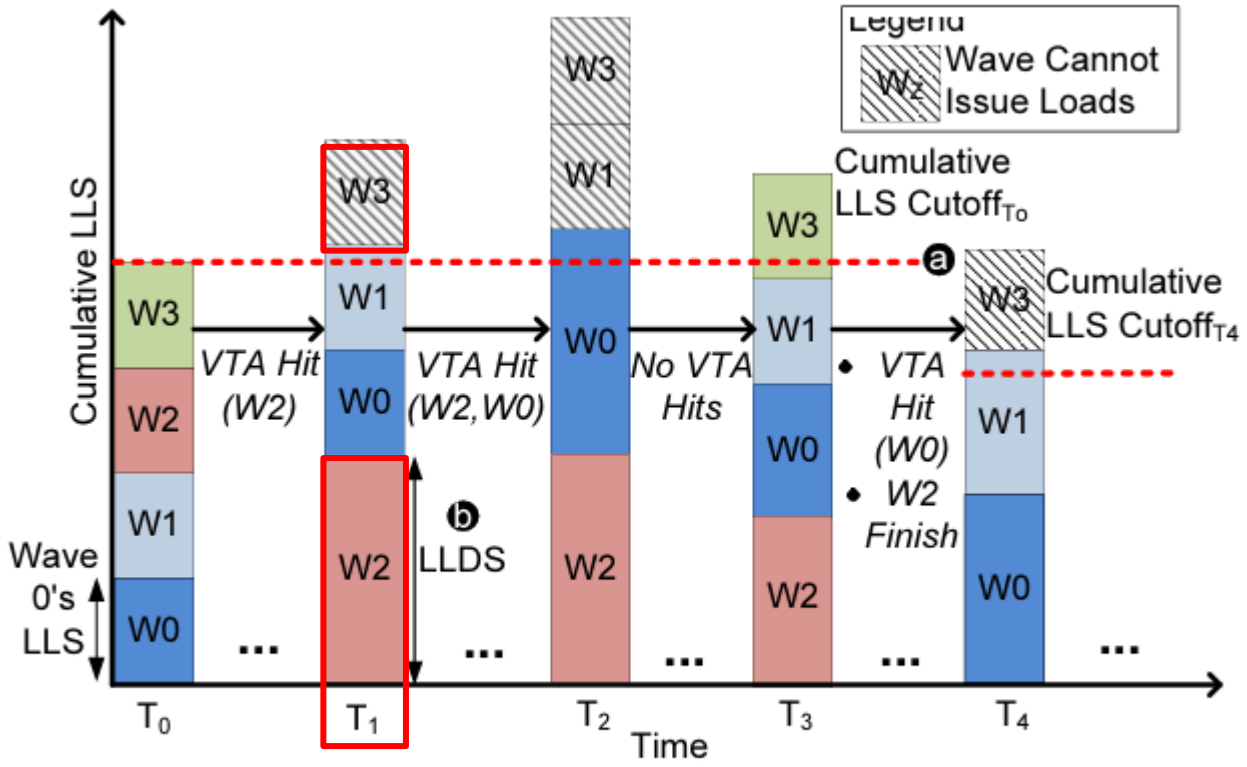


Figure 6: Locality scoring system operation example. LLS=*lost-locality score*, LLDS=*lost-locality detected score*

진행 현황

CCWS - Lost-Locality Detected(LLDS)

$$LLDS = \frac{VTAHits_{Total}}{InstIssued_{Total}} \cdot K_{THROTTLE} \cdot CumLLSCutoff$$

진행 현황

스케줄러 성능을 측정하기 위한 워크로드

Table 2: GPU Compute Benchmarks (CUDA and OpenCL)

Highly Cache Sensitive (HCS)			
Name	**Abbr.**	**Name**	**Abbr.**
BFS Graph Traversal [9]	BFS	Kmeans [9]	KMN
Memcached [16]	MEMC	Garbage Collection [5, 36]	GC
Moderately Cache Sensitive (MCS)			
Name	**Abbr.**	**Name**	**Abbr.**
Weather Prediction [9]	WP	Streamcluster [9]	STMCL
Single Source Shortest Path [4]	SSSP	CFD Solver [9]	CFD
Cache Insensitive (CI)			
Name	**Abbr.**	**Name**	**Abbr.**
Needleman-Wunsch [9]	NDL	Back Propagation [4]	BACKP
Speckle Red. Anisotropic Diff. [9]	SRAD	LU Decomposition [9]	LUD

진행 현황

스케줄러 성능을 측정하기 위한 워크로드

- LRR
- GTO
- 2LVL-GTO
- Best-SWL
- CCWS
- <Scheduler>-BEL

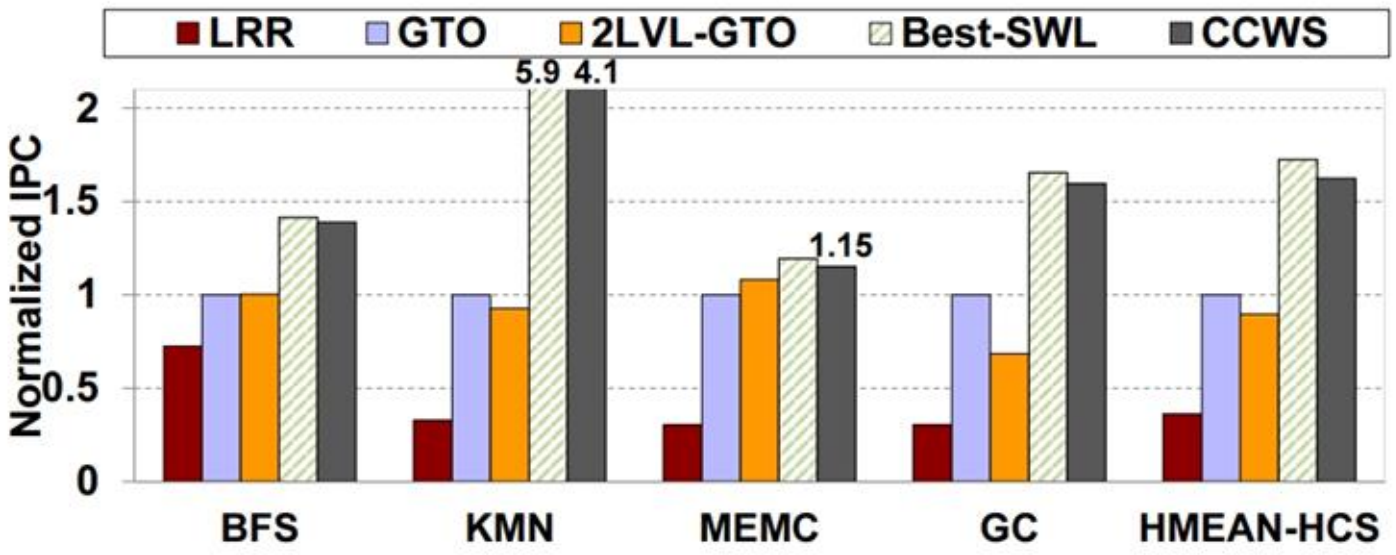


Figure 7: Performance of various schedulers and replacement policies for the highly cache-sensitive benchmarks. Normalized to the GTO scheduler.

진행 현황

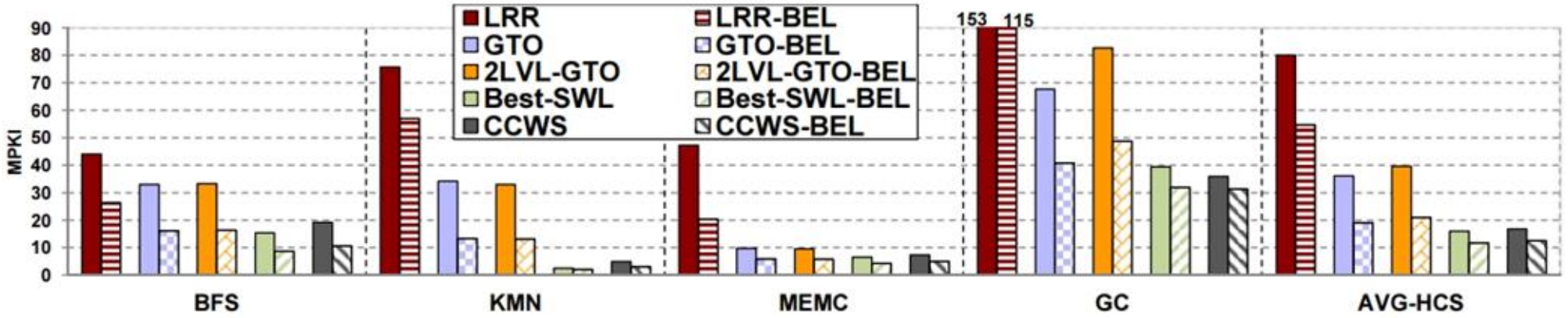


Figure 8: MPKI of various schedulers and replacement policies for the highly cache-sensitive benchmarks.

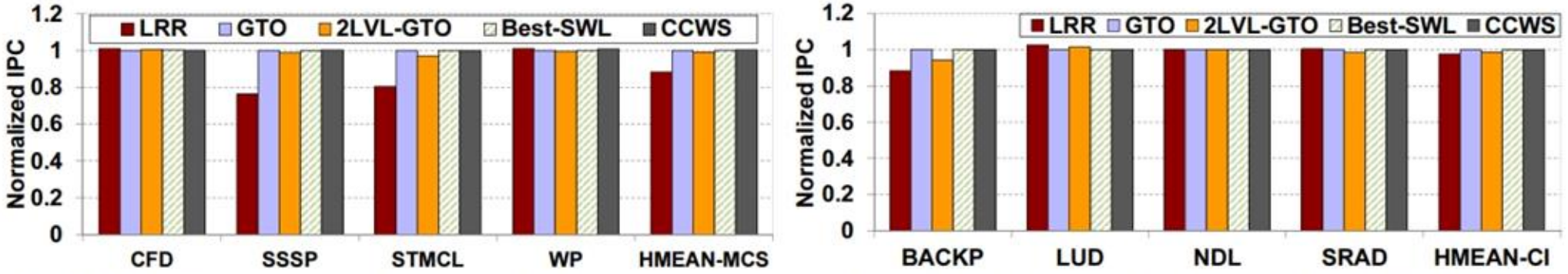


Figure 9: Performance of various schedulers and replacement policies for moderately cache-sensitive (left) and cache-insensitive (right) benchmarks. Normalized to the GTO scheduler.

진행 현황

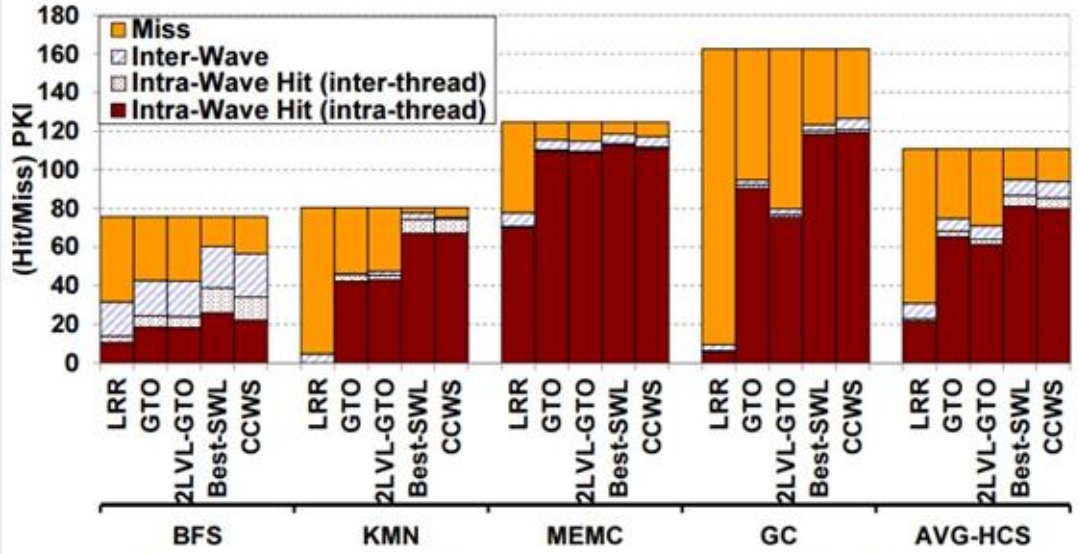


Figure 11: Breakdown of L1D misses, intra-wavefront locality hits (broken into intra-thread and inter-thread) and inter-wavefront locality hits per thousand instructions for highly cache-sensitive benchmarks. The configuration from Section 5.1 is used.

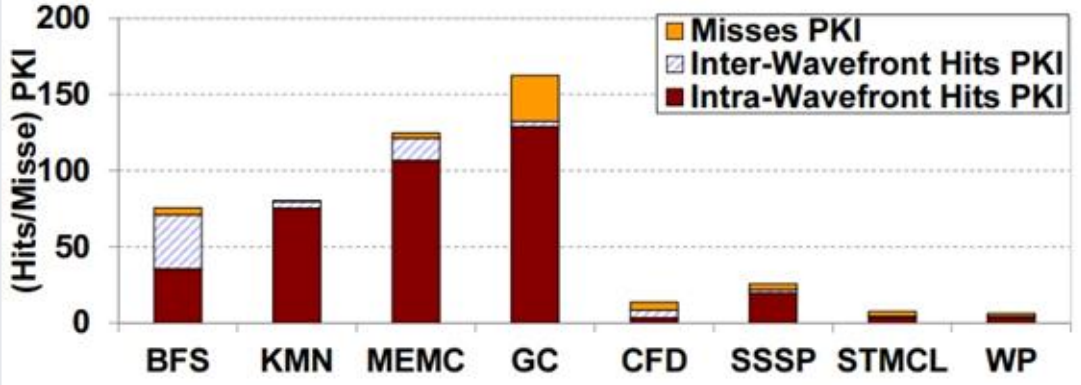


Figure 12: Breakdown of L1D misses, intra-wavefront locality Hits and inter-wavefront locality PKI using an unbounded L1 cache with 128 byte cache lines.

진행 현황

번외: **input size 변화에 따른 GTO vs CCWS**

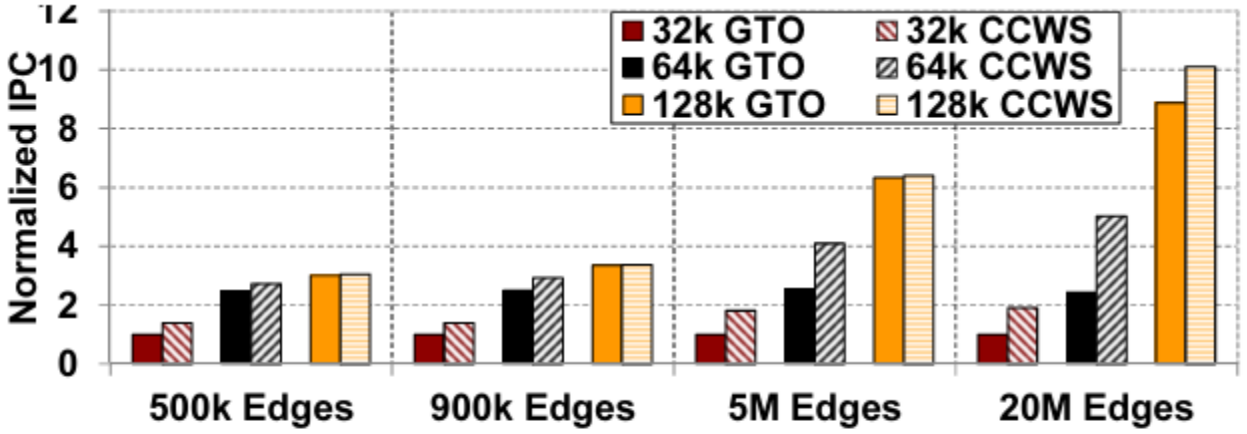


Figure 15: Performance of CCWS on BFS with different graph sizes when varying the L1D cache size and scheduler choice. Normalized to the GTO scheduler with a 32k L1D. The VTA size is 16 entries per wavefront for all instances of CCWS.

진행 현황

번외: VTA 변화에 따른 CCWS 성능 변화:

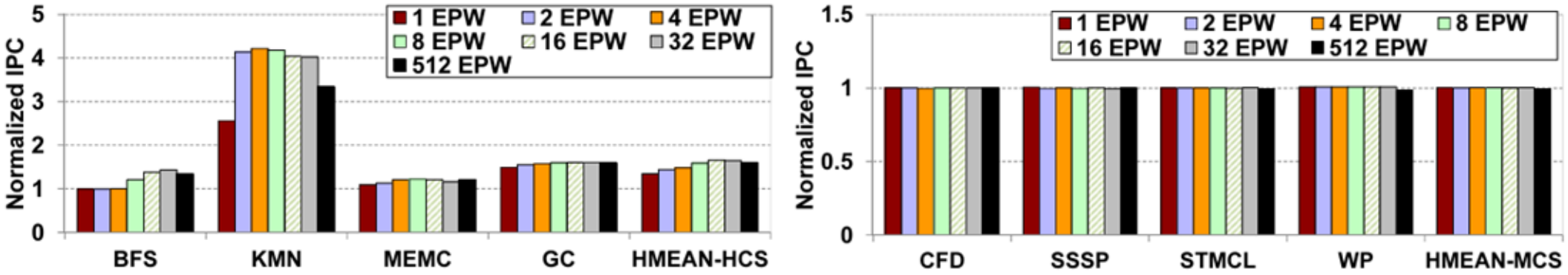


Figure 13: Performance of CCWS at various victim tag array sizes. Normalized to the GTO scheduler. EPW=Entries per Wavefront. EWP 1-4 are 1-4 set associative respectively. All other victim tag arrays are 8-way set associative.

[SECTION]

[GTO/RR Scheduler 구현]

진행 현황

What we did

```
namespace {

// Choose one schedule policy here for simx:
// WarpSchedulePolicy::Static
// WarpSchedulePolicy::RR
// WarpSchedulePolicy::GTO
constexpr WarpSchedulePolicy kDefaultSchedulePolicy = WarpSchedulePolicy::Static;

} // namespace
```

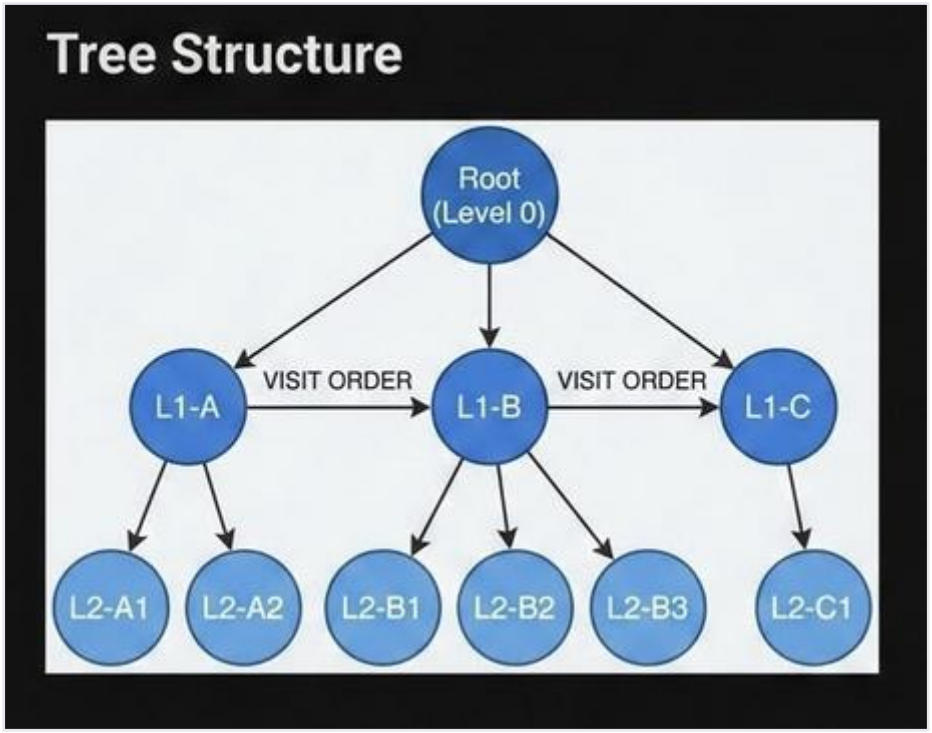
- GTO, RR 구현.
- 취사 선택 가능.

Configuration

<configuration 설정값

항목	값
cores	1
warps	16
threads	16
ICACHE	16KB (기본값)
ICACHE_WAYS	4 (기본값)
ICACHE_MSHR	16 (기본값)
DCACHE	128KB
DCACHE_WAYS	8
DCACHE_MSHR	16 (기본값)
L2	256KB
L2_WAYS	16

<bfs>



벤치마킹 결과 : BFS

지표	RR	GTO	변화
IPC	1.908	2.026	+6.2%
cycles	114,452	107,814	-5.8%
load latency	48 cycles	44 cycles	-8.3%
scheduler idle	87%	86%	-1%p
ibuffer stalls	38%	55%	+17%p
scoreboard stalls	85%	85%	0
└ lsu	95%	94%	-1%p

- GTO와 RR이 정상적으로 구현되었음.
- Load Latency 8.3% 감소
- Greedy : I-buffer Stall 증가 (+17%p)
- scheduler idle : 87%
- LSU scoreboard Stall : 95%

문제점 분석 : Configuration

Vortex MSHR 구성

캐시	위치	MSHR size
ICache (L1 instruction)	소켓별	16
DCache (L1 data)	소켓별	16
L2 Cache	클러스터별	16
L3 Cache	프로세서 전체	16

1. memory bandwidth 문제
   a. 스케줄러가 관여할 시간이 없음
   b. configuration 조정 필요

1. Irregular memory access 문제
   a. bfs workload의 특성 - uncoalesced
   b. cache miss 방지를 위한 정책 필요

[SECTION]

[논문 리뷰]

CAWA: Coordinated Warp Scheduling and Cache Prioritization for Critical Warp Acceleration of GPGPU Workloads

Introduction

파이프라인 지연을 유발하는 Critical Warp 식별 및 최적화

CPL (Criticality Prediction Logic)

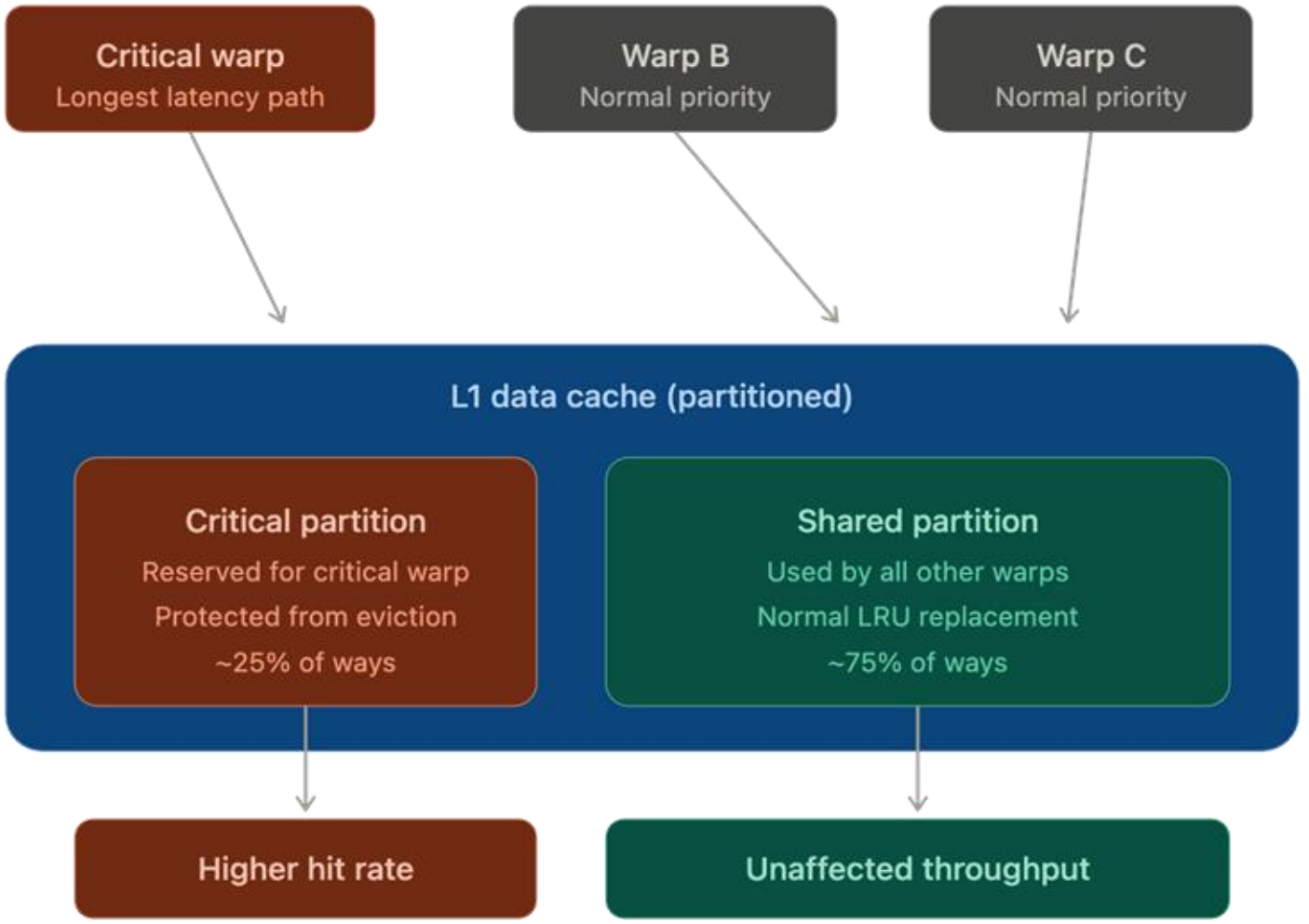
$$nCriticality = nInst * w.CPI_{avg} + nStall;$$

- Criticality 카운터로 런타임중에 Ciritical warp를 예측.
  - nInst: 워크로드 불균형 및 Branch divergence 로 인해 발생한 명령어 수 격차
  - w.CPIavg: 워프당 평균 CPI
  - nStall: resource contention 등으로 인해 발생한 Stall cycle

gCAWS (greedy Criticality-Aware Warp Scheduler)

Critical Warp를 더 자주 , 더 길게 실행

CACP (Criticality-Aware Cache Prioritization)



CACP (Criticality-Aware Cache Prioritization)

CCBP

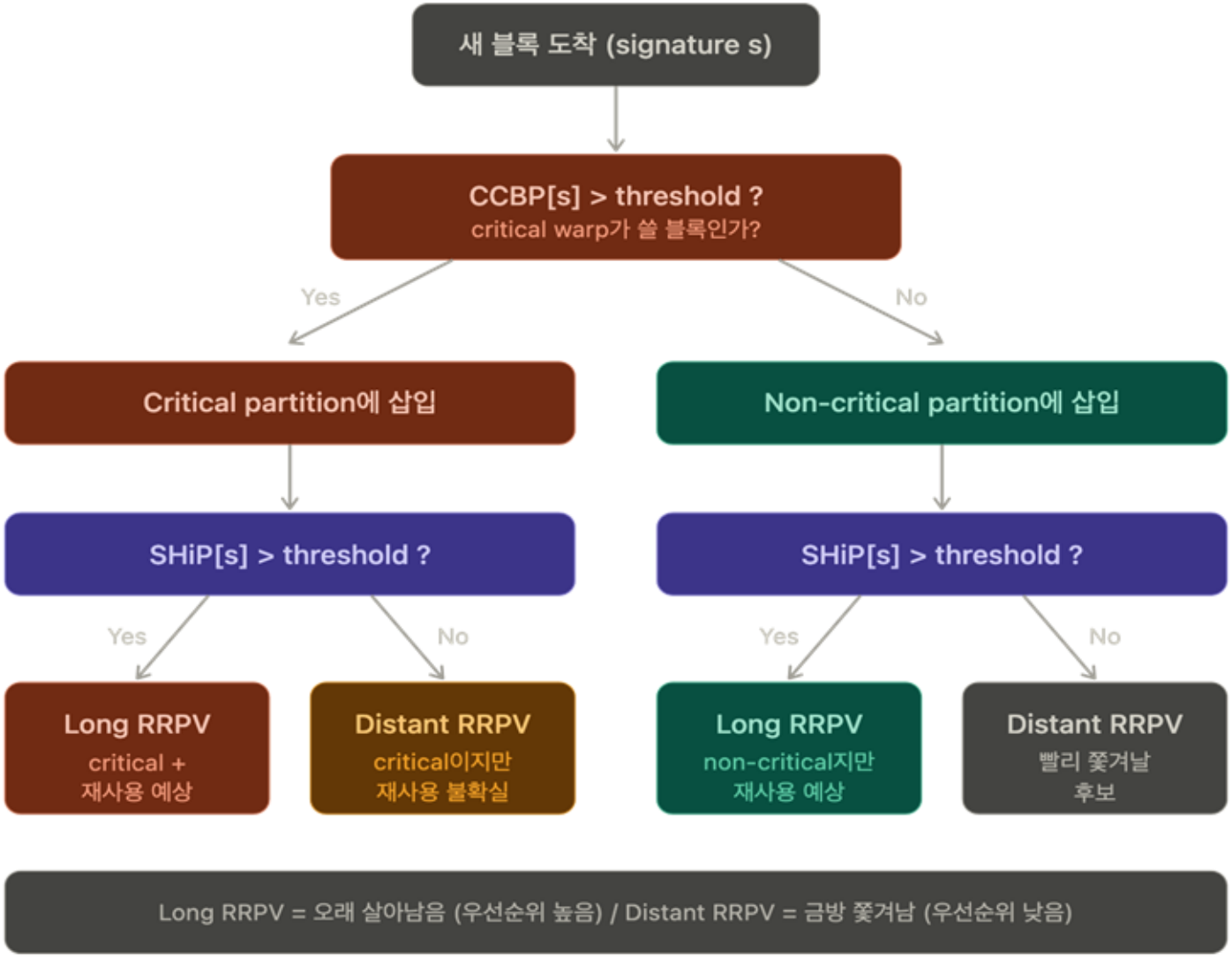
Critical Cache Block Predictor
캐시 라인이 critical warp에 사용될지 예측

+

SHiP

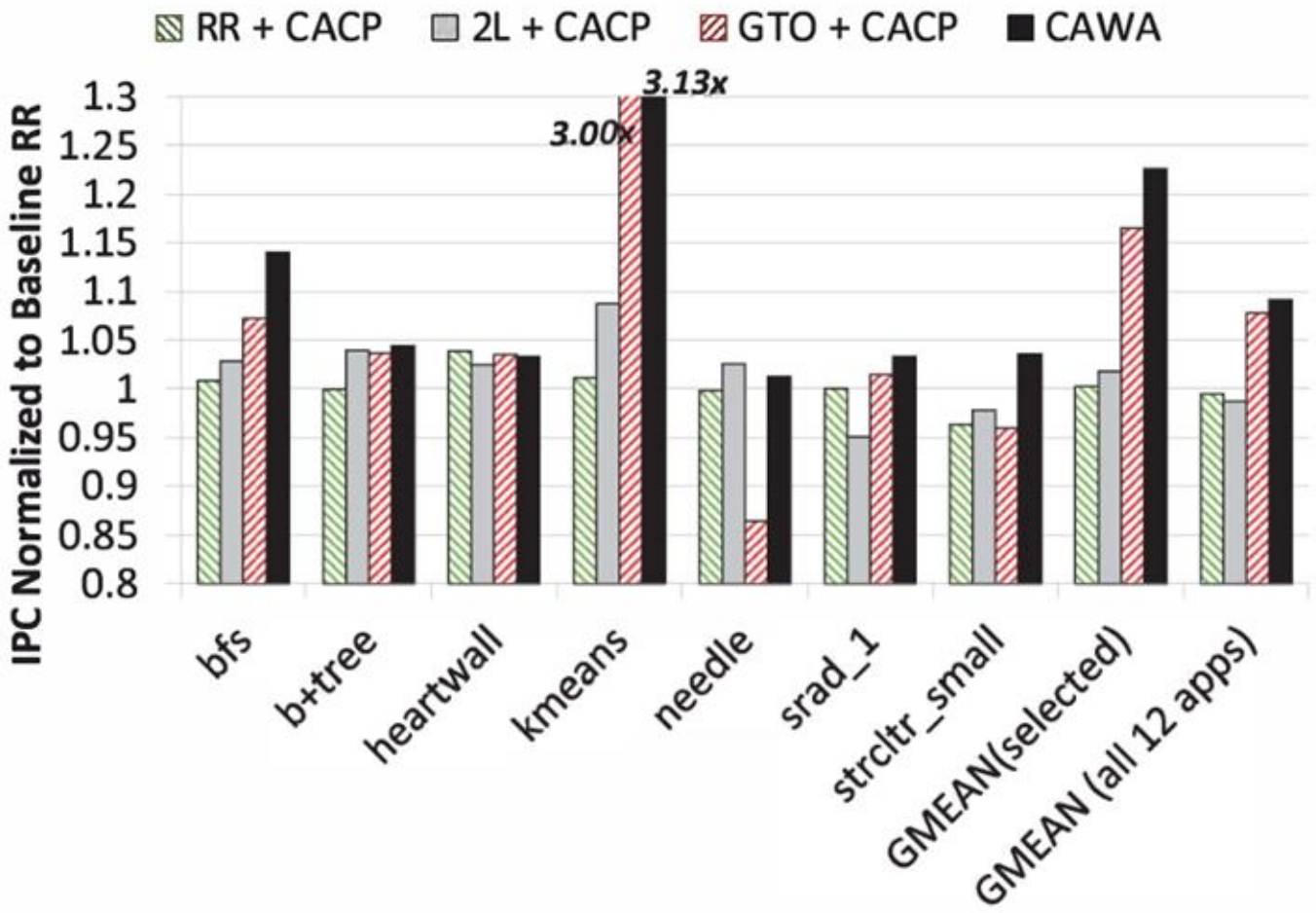
Signature-based Hit Predictor
캐시 라인의 재사용 패턴을 예측

CACP (Criticality-Aware Cache Prioritization)



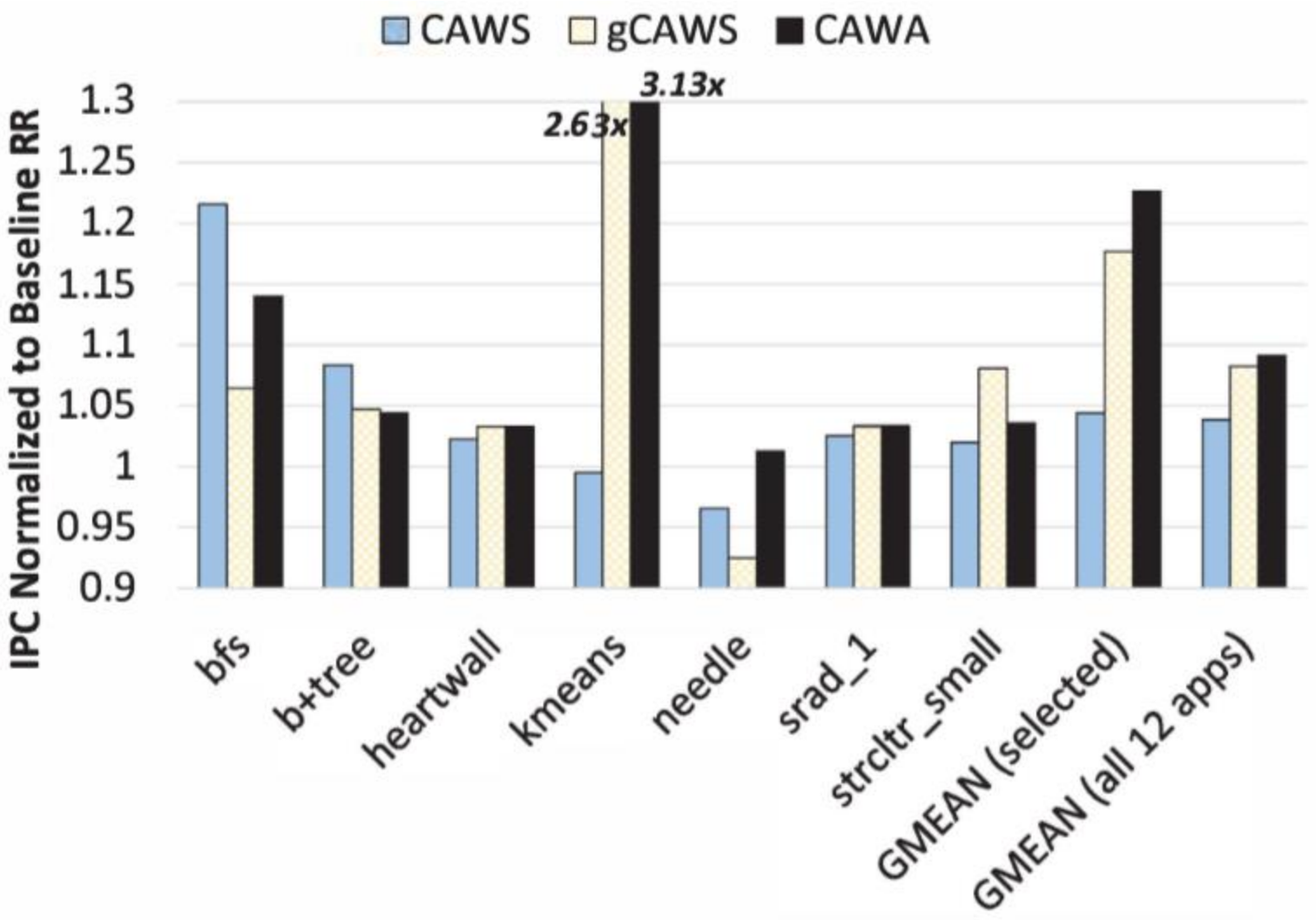
- signature = `PC[7:0] XOR addr[7:0]`
- CCBP : 어느 **파티션**에 넣을지 결정
- SHiP : 파티션 내 우선순위 결정

CAWA



- 대부분의 workload에서의 성능 향상
  - baseline(RR) 대비 **가장 높은 IPC** 향상 달성.
  - kmeans 의 경우 약 3배 향상

논문 의의



- bfs의 irregular memory access 완화
- 스케줄러(gCAWS) + 캐시(CACP) –> 최적화

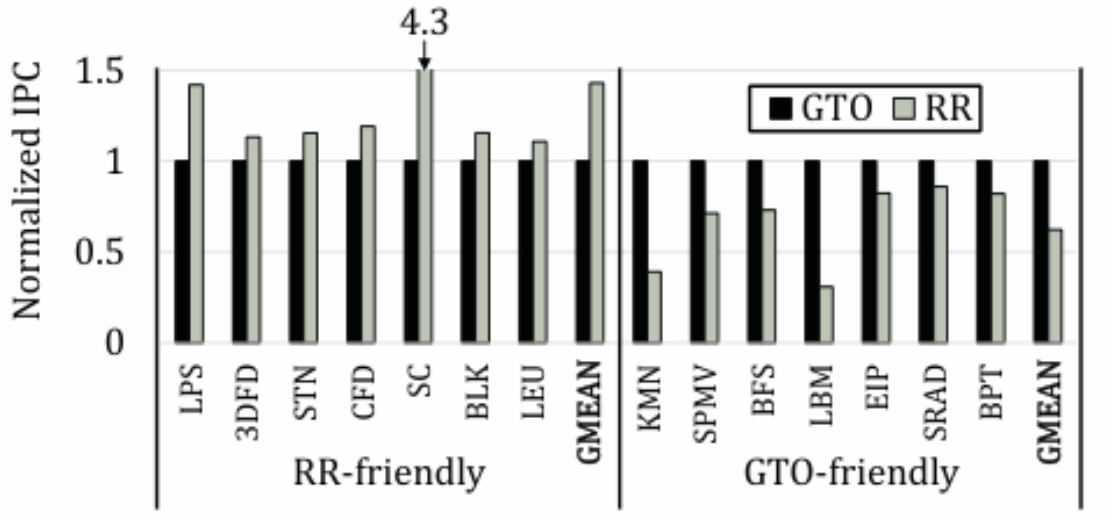
[SECTION]

[논문 리뷰]

iPAWS : Instruction-Issue Pattern-based
Adaptive Warp Scheduling for GPGPUs

Paper review: iPAWS

Instruction-Issue Pattern based Adaptive Warp Scheduling



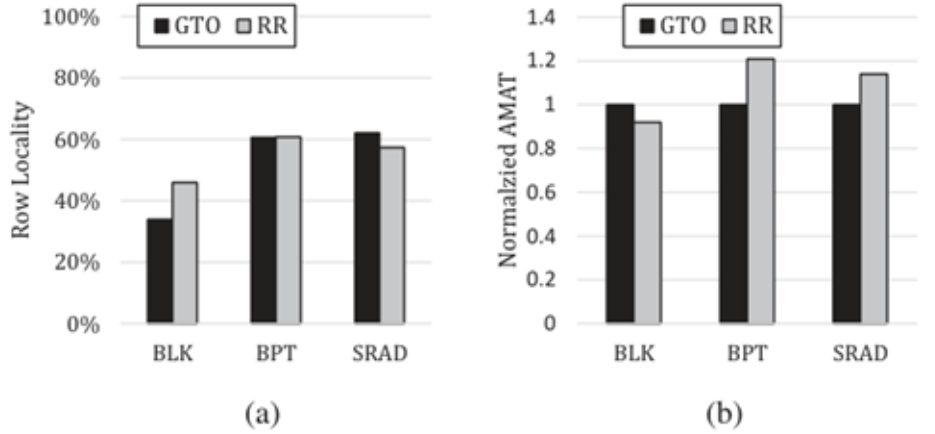
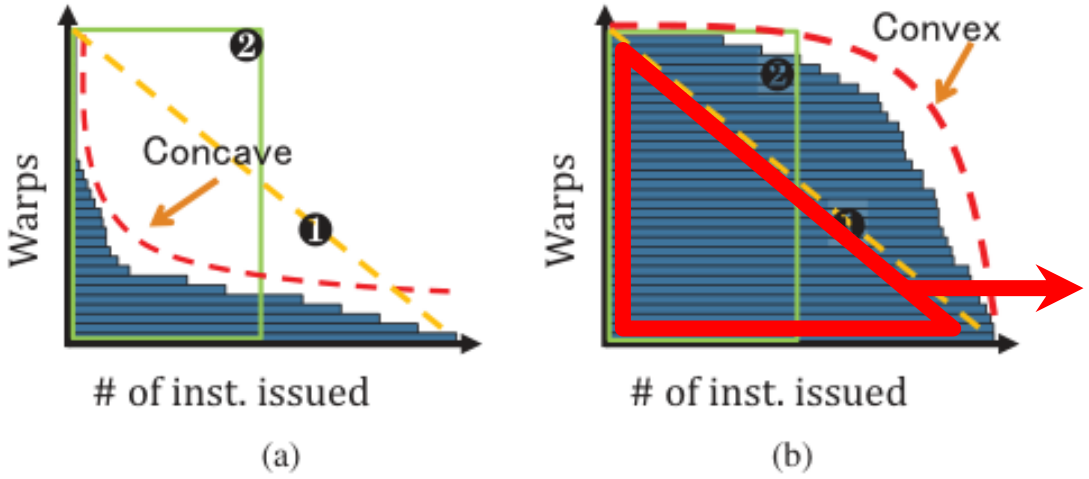


Figure 5: (a) DRAM row-buffer locality and (b) Average memory latency for workloads whose performance is not determined by cache locality but DRAM access pattern.

GTO, RR중 워크로드마다 선호되는 정책이 존재
그러나 Cache hit Rate 외 병목 존재

- DRAM 접근 패턴
- Row-buffer locality
- warp간 load imbalance

Paper review: iPAWS



$$inst_{thr} = max_num_warp \times \left(\frac{inst_{max}}{2}\right)$$

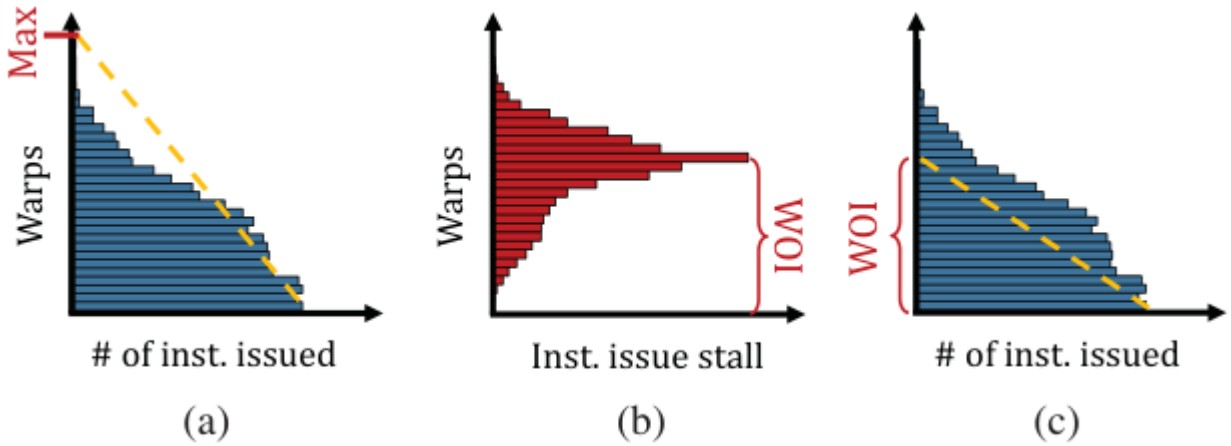
Instruction Issue pattern을 통해 선호 정책을 파악
GTO-Friendly한 워크로드는 이슈 패턴이 concave
RR-Friendly한 워크로드는 이슈 패턴이 convex

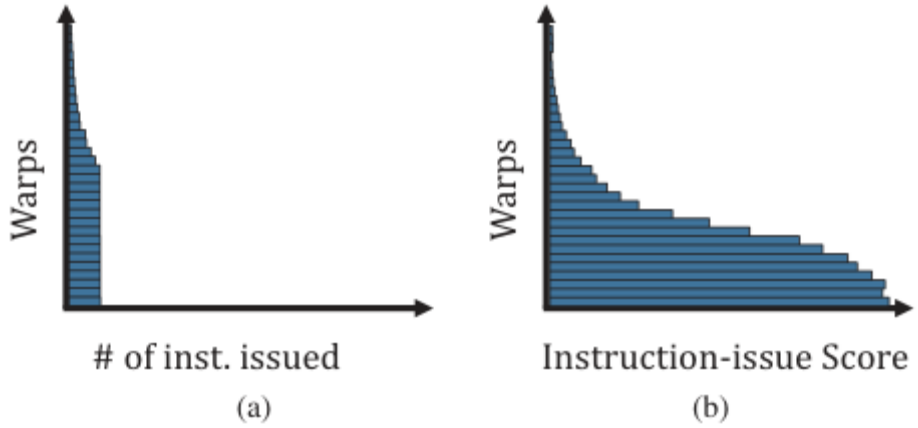
첫 번째 워프의 실행이 완료된 시점에서 이슈 패턴을 파악

Paper review: iPAWS

$$inst_{thr} = max_num_warp \times (\frac{inst_{max}}{2})$$

$$iscore_i = inst_i + btime_i$$
$$iscore_{thr} = \mathrm{W} \times (\frac{iscore_{max}}{2})$$
$$iscore_{sum} = \sum_{i=0}^{W} (iscore_i)$$

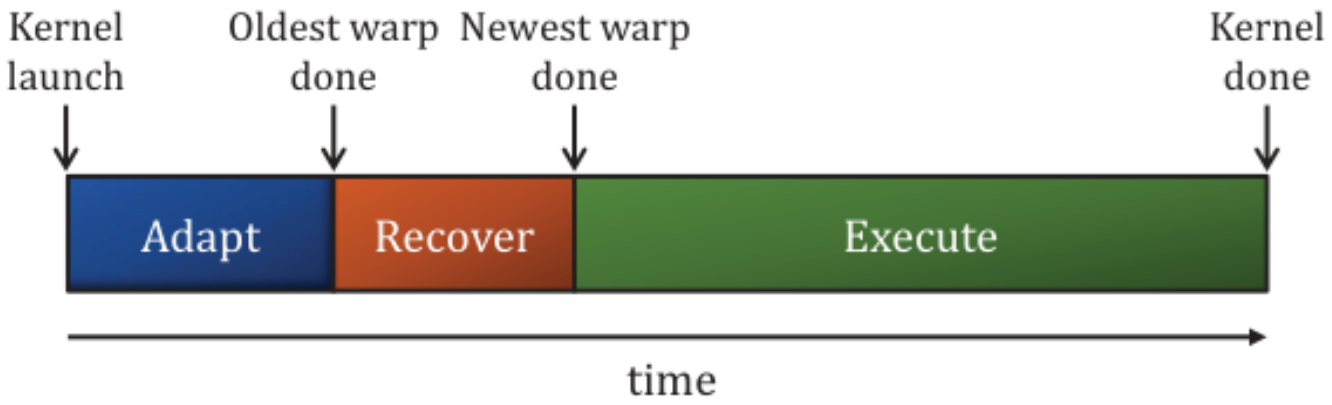




convex-concave 파악 시 inst_max 사용 시 불일치 발생
(b)와 같이 oldest이지만 issue가 안된 횟수 파악
가장 큰 값을 가진 워프를 WOI(Warp of Interest) 기준으로 삼음

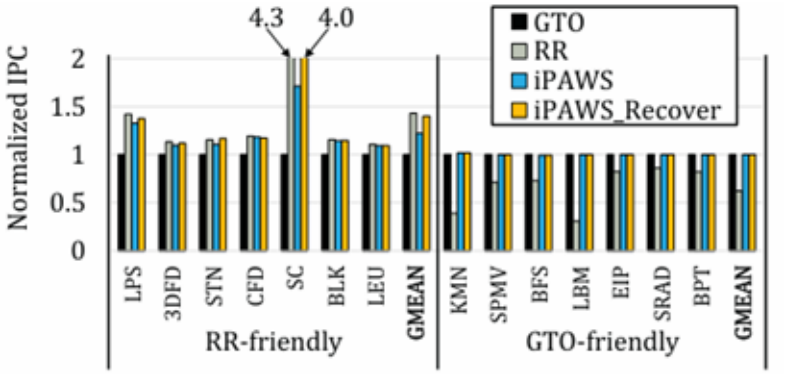
barrier synchronization 존재 시 워프가 고르게 분포
왜곡을 최소화하기 위해
barrier로 인한 stall 시간을 score에 합산

Paper review: iPAWS

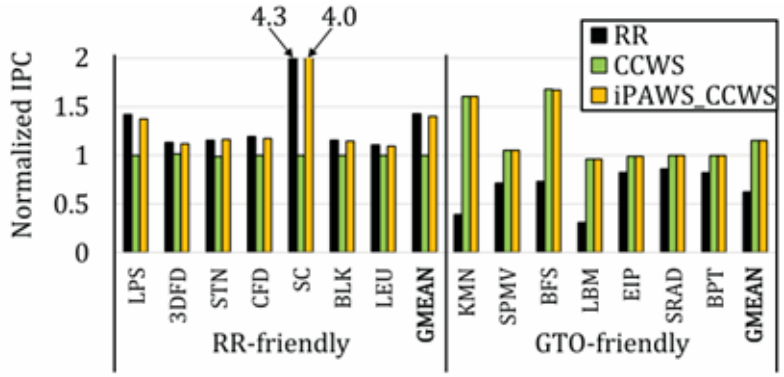


GTO로 판단되면 계속 실행
RR로 판단되면 Recover Phase에서 least instruction-issued 워프 우선, 균형을 맞춤

Paper review: iPAWS



RR-Friendly workload가 GTO 대비 약 70% 성능 향상



RR-GTO 뿐만이 아닌 CCWS, MASCAR 등 타 워프 스케줄러와 조합 가능
CCWS 단독 사용보다 성능 평균 40% 향상
범용 GPU로서의 성능 향상